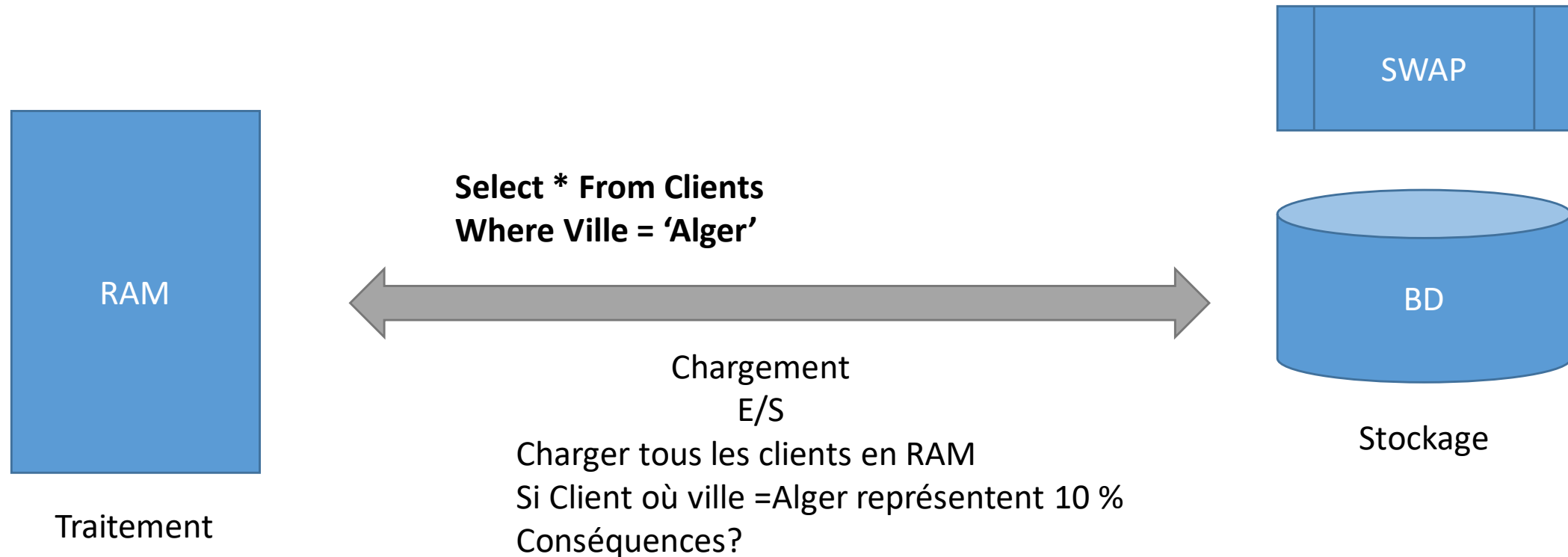


Optimisation des requêtes

Introduction



❑ Évaluation d'une requête SQL:

- Analysée syntaxiquement, ensuite transformée en plan d'évaluation.
- **Plan d'évaluation:** Arbre d'opérations de l'algèbre relationnelle avec un choix d'algorithme pour chaque opération.

❑ Deux problématiques importantes dans l'optimisation:

- Énumération des plans alternatifs pour une requête
- Estimation des coûts de ces plans et choix de celui estimé être le moins cher

Rules Based Optimisation

Requête (SQL)



Traduction

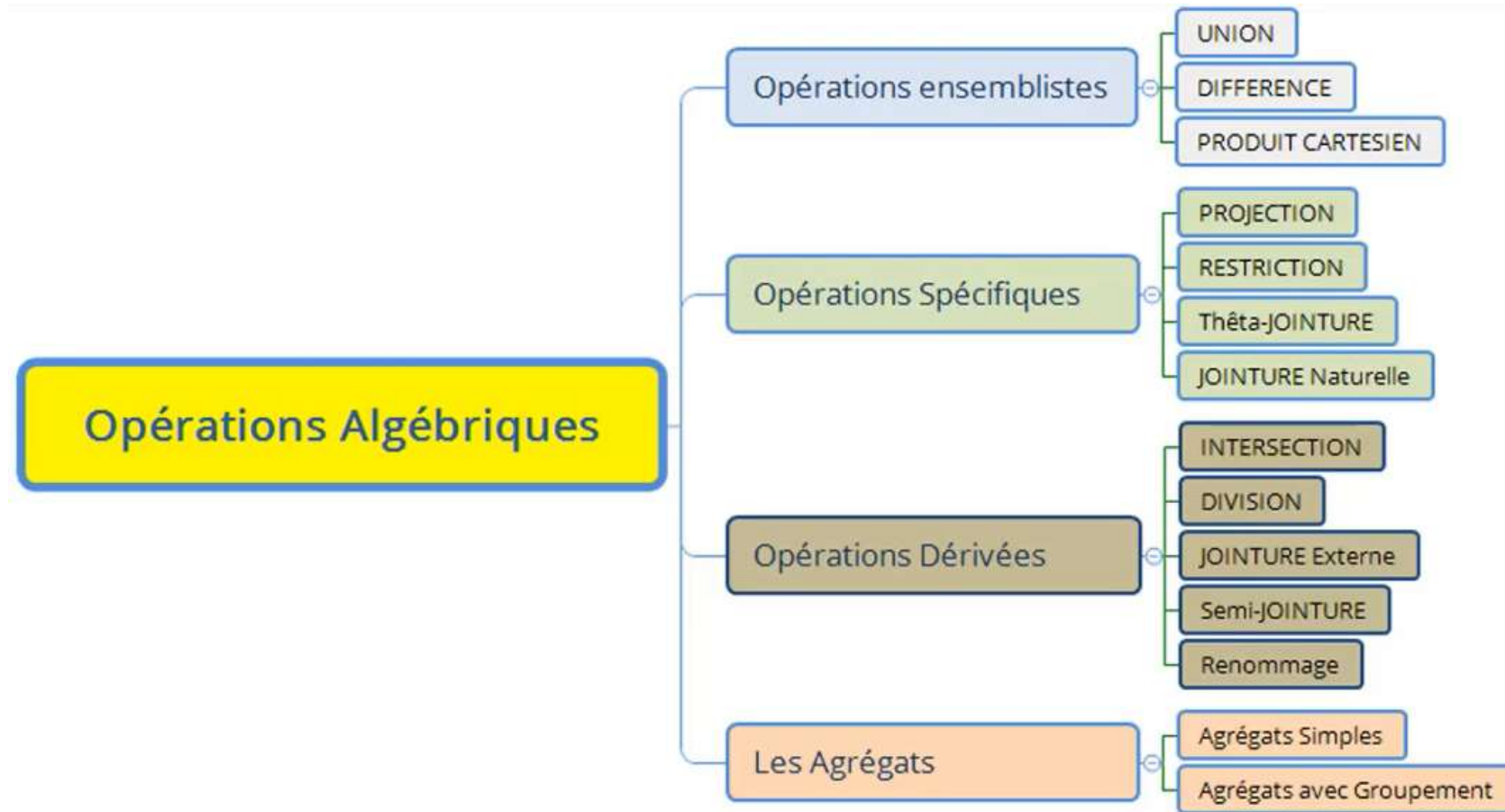
Requête en algèbre



Transformation en fonction des
propriétés des opérateurs

Plan optimal

Opérations algébriques



Opérations algébriques

- L'algèbre relationnelle est une collection d'opérations formelles qui agissent sur des relations et produisent des relations, en appliquant les opérations de l'algèbre relationnelle sur une seule table ou plusieurs tables (relations) selon le type de l'opérateur, nous obtenons toujours une relation
- Une opération algébrique est une opération mathématique appliquée sur des relations produit une relation, elles sont utilisées afin de former des requêtes

Requête algébrique

Cette requête a la forme suivante, et à chaque requête algébrique correspond un arbre

$$R1 \leftarrow \sigma_{\text{taille}=32}(\text{VESTE})$$

$$R2 \leftarrow \sigma_{\text{couleur=rouge}}(\text{VESTE})$$

$$R3 \leftarrow R1 \cap R2$$

$$R4 \leftarrow \pi_{\text{marque}}(R3)$$

Arbre algébrique

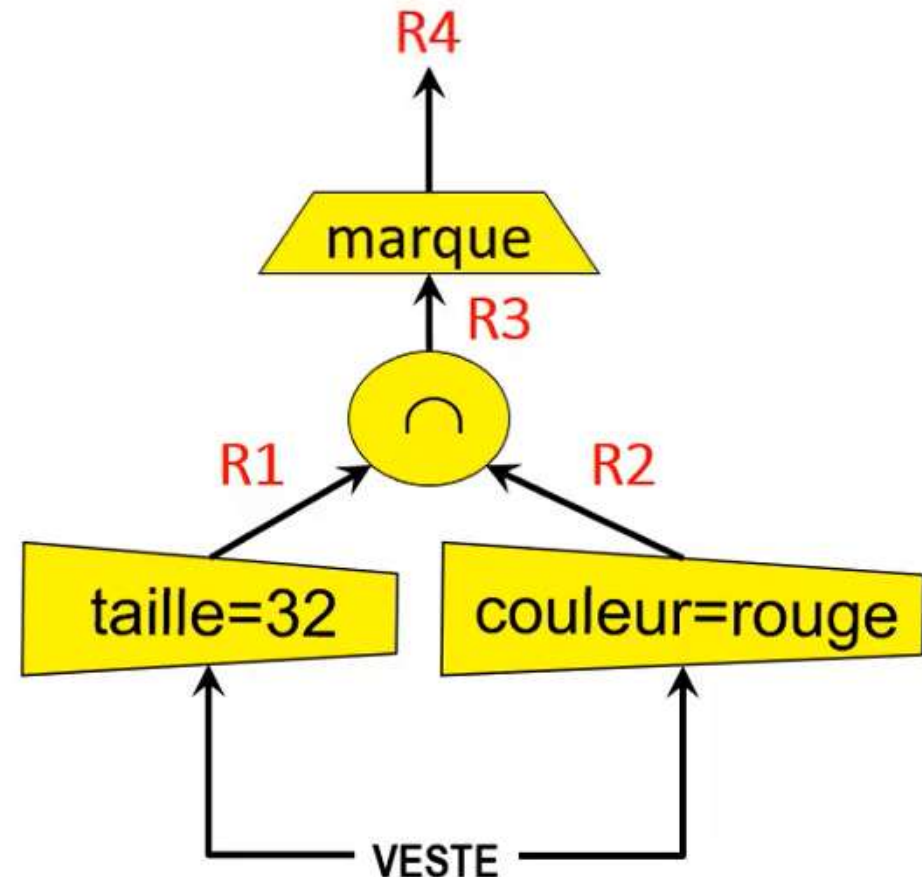
À chaque requête correspond un arbre algébrique dont les nœuds représentent les opérations et les arcs des relations

$$R1 \leftarrow \sigma_{\text{taille}=32}(\text{VESTE})$$

$$R2 \leftarrow \sigma_{\text{couleur}=rouge}(\text{VESTE})$$

$$R3 \leftarrow R1 \cap R2$$

$$R4 \leftarrow \pi_{\text{marque}}(R3)$$



Les opérations ensemblistes

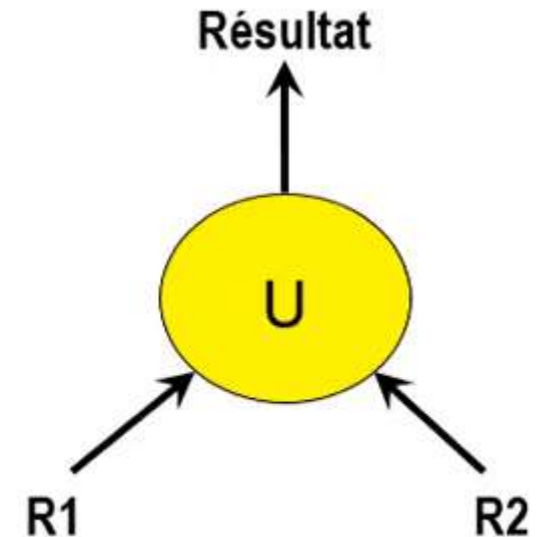
- Union
- Différence
- Produit cartésien

Union

L'union s'applique à deux relations ayant la même structure, lorsque l'on veut afficher dans un même résultat, deux résultats provenant de deux relations différentes, L'union élimine les doublons

$$R_1 \cup R_2$$

$$UNION(R_1, R_2)$$



Union

Client_Alger		
N°	Nom	TEL
1	ALI	021324354
2	AHMED	021554354

Client_Bouira		
N°	Nom	TEL
2	AHMED	021554354
3	MALIK	09876542

Clients		
N°	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542

Différence

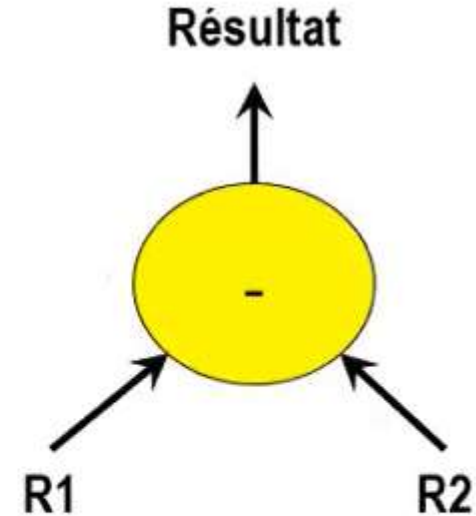
La différence s'applique à deux relations ayant la même structure , lorsque l'ont veut afficher un résultat n'ayant pas effectué un acte donné

$$R_1 - R_2$$

$$DIFFERENCE(R_1, R_2)$$

$$REMOVE(R_1, R_2)$$

$$MINUS(R_1, R_2)$$



Différence

Clients		
N°	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

Clients_Actif_2020		
N°	Nom	TEL
2	AHMED	021554354
3	MALIK	09876542

Les client inactifs depuis 2020

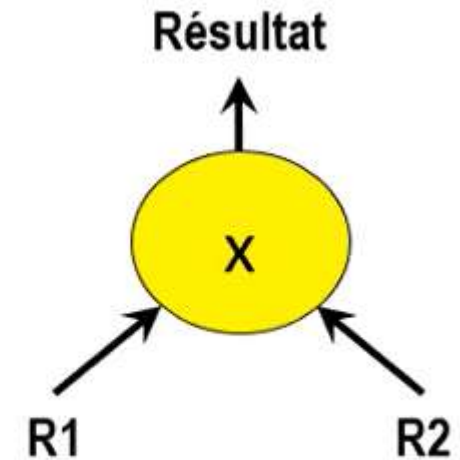
Clients_Inactif_2020		
N°	Nom	TEL
1	ALI	021324354
4	AHMED	021554354

Produit cartésien

Le produit cartésien n'a aucune signification seul il est souvent combiné avec d'autres opérations pour aboutir à un résultat donné, il n'y a aucune condition sur la structure des données, la structure du résultat est la structure de la relation 1 combinée avec la structure de la relation 2 (chaque tuple de la relation 1 est combiné avec les tuples de relation 2)

$$R_1 \times R_2$$

$$PRODUCT(R_1, R_2)$$



Produit cartésien

R1		
N°	Nom	Age
1	ALI	20
2	AHMED	40

R2	
Cmd	Date
2	23/02/07
3	01/01/14

$R_1 \times R_2$				
N°	Nom	AGE	Cmd	Date
1	ALI	20	2	23/02/07
1	ALI	20	3	01/01/14
2	AHMED	40	2	23/02/07
2	AHMED	40	3	01/01/14

Les opérations spécifiques

- Projection
- Restriction
- Thêta-jointure
- Jointure naturelle

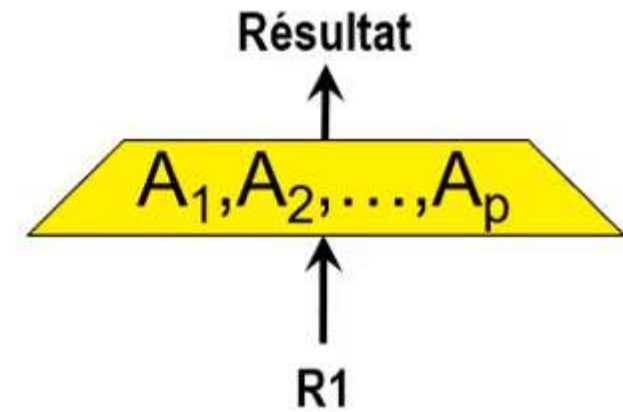
Projection

Limiter le nombre de colonnes à afficher dans le résultat, elle s'applique sur une seule relation

$$\Pi_{A_1, A_2, \dots, A_p}(R)$$

$$R[A_1, A_2, \dots, A_p]$$

$$\text{PROJECT}(R, A_1, A_2, \dots, A_p)$$



Projection

R		
N°	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	ALI	021565432
4	MALIK	023232343

Project (R,Nom)
Nom
ALI
AHMED
ALI
MALIK

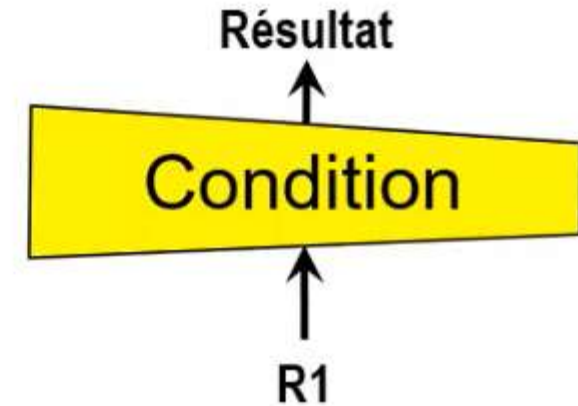
Restriction

Effectuer des filtres sur les données d'une table

$\sigma_{condition} (R)$

R[Condition]

RESTRICT(R,Condition)



Att <Op> Att
Att <Op> Val
=, <, <=, >, >=, <>

Restriction

R		
N°	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	ALI	034456543

Nom='ALI'

$\sigma_{nom='ALI'}(R)$		
N°	Nom	TEL
1	ALI	021324354
3	ALI	034456543

Jointure (thêta-jointure)

Le lien se fait avec la clé étrangère et clé primaire, la colonne est affichée en double

Etudiants

student_id	name	age
1	Akon	17
2	Bkon	18
3	Ckon	17
4	Dkon	18

Notes



student_id	subject_id	marks
1	1	98
1	2	78
2	1	76
3	2	88

student_id	name	age	student_id	subject_id	marks
1	Akon	17	1	1	98
1	Akon	17	1	2	78
2	Bkon	18	2	1	76
3	Ckon	17	3	2	88

Jointure (thêta-jointure)

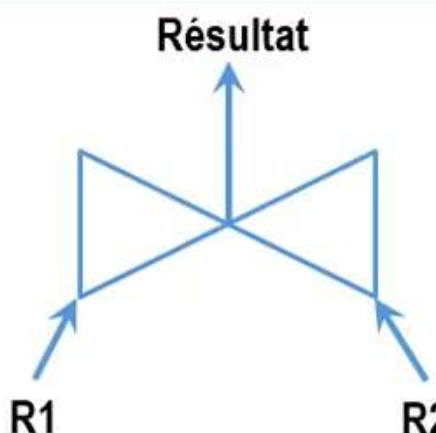
R1 Condition R2	Présentation																														
Join(R1,R2,Cond)	Join(Etudiants,Notes,Etudiants.Student_Id=Notes.Student_Id)																														
Résultat Ai Bi Condition R1 R2	<table><tr><th>student_id</th><th>name</th><th>age</th><th>student_id</th><th>subject_id</th><th>marks</th></tr><tr><td>1</td><td>Akon</td><td>17</td><td>1</td><td>1</td><td>98</td></tr><tr><td>1</td><td>Akon</td><td>17</td><td>1</td><td>2</td><td>78</td></tr><tr><td>2</td><td>Bkon</td><td>18</td><td>2</td><td>1</td><td>76</td></tr><tr><td>3</td><td>Ckon</td><td>17</td><td>3</td><td>2</td><td>88</td></tr></table>	student_id	name	age	student_id	subject_id	marks	1	Akon	17	1	1	98	1	Akon	17	1	2	78	2	Bkon	18	2	1	76	3	Ckon	17	3	2	88
student_id	name	age	student_id	subject_id	marks																										
1	Akon	17	1	1	98																										
1	Akon	17	1	2	78																										
2	Bkon	18	2	1	76																										
3	Ckon	17	3	2	88																										
Ai ∈ à Delta(R1) Bi ∈ à Delta(R2)																															

Thêta-jointure

$$R1 \bowtie_{\text{Condition}} R2 \equiv \sigma_{\text{condition}} (R1 \times R2)$$

Jointure (jointure naturelle)

la commande NATURAL JOIN permet de faire une jointure naturelle entre 2 tables. Cette jointure s'effectue à la condition qu'il y ai des colonnes du même nom et de même type dans les 2 tables. Le résultat d'une jointure naturelle est la création d'un tableau avec autant de lignes qu'il y a de paires correspondant à l'association des colonnes de même nom

R1 ⋈ R2	Présentation																									
Join(R1,R2)																										
Résultat 	Join(Etudiants,Notes) <table><tr><th>student_id</th><th>name</th><th>age</th><th>subject_id</th><th>marks</th></tr><tr><td>1</td><td>Akon</td><td>17</td><td>1</td><td>98</td></tr><tr><td>1</td><td>Akon</td><td>17</td><td>2</td><td>78</td></tr><tr><td>2</td><td>Bkon</td><td>18</td><td>1</td><td>76</td></tr><tr><td>3</td><td>Ckon</td><td>17</td><td>2</td><td>88</td></tr></table>	student_id	name	age	subject_id	marks	1	Akon	17	1	98	1	Akon	17	2	78	2	Bkon	18	1	76	3	Ckon	17	2	88
student_id	name	age	subject_id	marks																						
1	Akon	17	1	98																						
1	Akon	17	2	78																						
2	Bkon	18	1	76																						
3	Ckon	17	2	88																						

Jointure par boucle imbriquée

Chaque ligne de la première table est comparée avec toutes les lignes de la deuxième table

Client		
Nclt	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

Véhicule	NV	Marque	Client
	2	Hyundai	4
	3	Fiat	1
	4	DACIA	4

JOIN(Client,Véhicule,Client.N°=Véhicule.Client)					
N°	Nom	TEL	NV	Marque	Client
1	ALI	021324354	3	Fiat	1
4	AHMED	021554354	2	Hyundai	4
4	AHMED	021554354	4	DACIA	4

Jointure par boucles imbriquées (JBI)

□ $R \bowtie_F S$

□ Principe:

- La première table est lue séquentiellement et de préférence stockée entièrement en mémoire
- Pour chaque tuple de R, il y a une comparaison avec les tuples de S

POUR chaque ligne IR de R

POUR chaque ligne IS de S

SI θ sur IR et IS est satisfait

Produire la ligne concaténée à partir de IR et IS

FINSI

FINPOUR

FINPOUR

+ Algorithme simple

Complexité: $(|R| + (|R| * |S|))$

Si la table extérieure tient en mémoire: une seule lecture des deux tables suffit.

Jointure par boucle imbriquée

- pour chaque tuple de R, il y a une comparaison avec chacun des tuples de S
- Le coût en nombre d'E/S: nombre de pages chargées pour pouvoir exécuter l'opération est : $|R| + |R| * |S|$
- $|R|$: nombre de pages de la table R
- $|S|$: nombre de pages de la table S

Jointure par tri fusion

Tri de la première table selon le numéro du client

Client		
Nclt	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

JOIN(Client,Véhicule,Client.N°=Véhicule.Client)					
NClT	Nom	TEL	NV	Marque	Client

Tri de R1 Selon NClT

Véhicule	NV	Marque	Client
	2	Hyundai	4
	3	Fiat	1
	4	DACIA	4

Jointure par tri fusion

Tri de la deuxième table selon le numéro du véhicule

Client		
Nclt	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

Véhicule	NV	Marque	Client
	3	Fiat	1
	2	Hyundai	4
	4	DACIA	4

```
JOIN(Client,Véhicule,Client.N°=Véhicule.Client)
```

Tri de R2 Selon
Client

Jointure par tri fusion

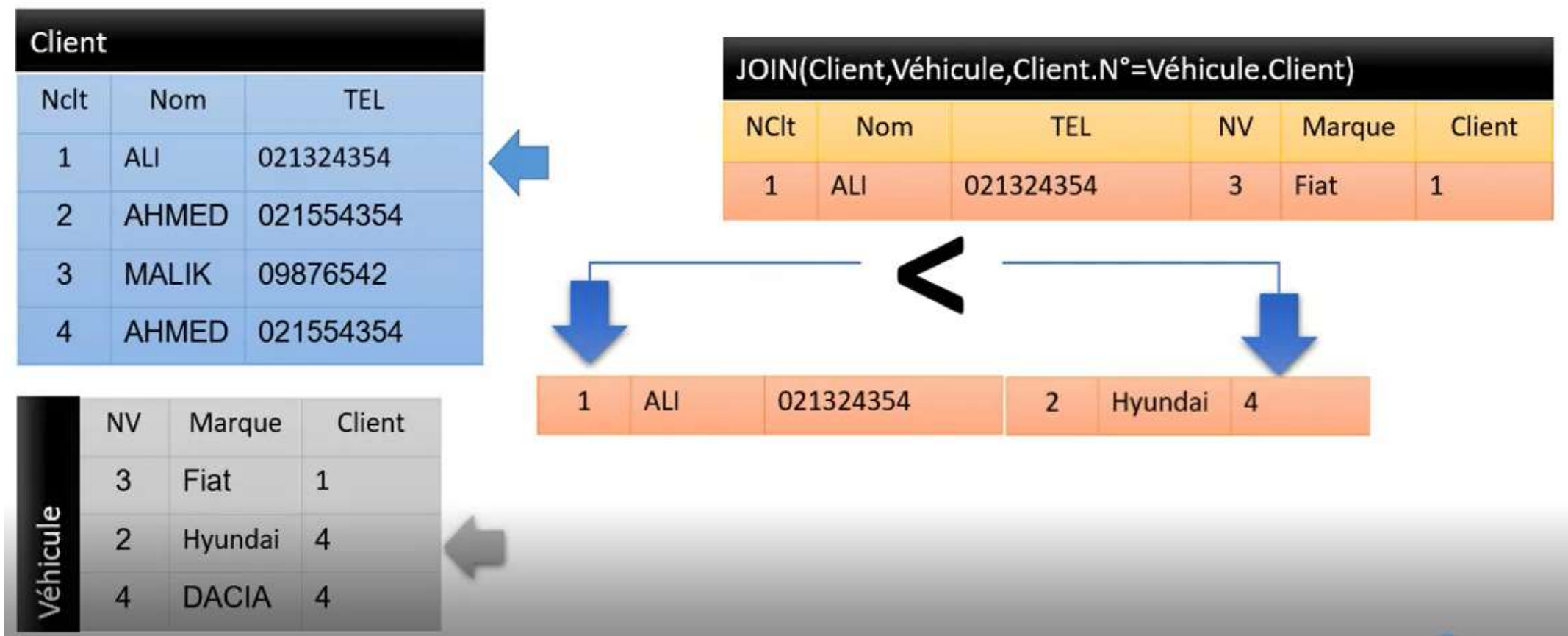
Client		
Nclt	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

Véhicule	NV	Marque	Client
	3	Fiat	1
	2	Hyundai	4
	4	DACIA	4

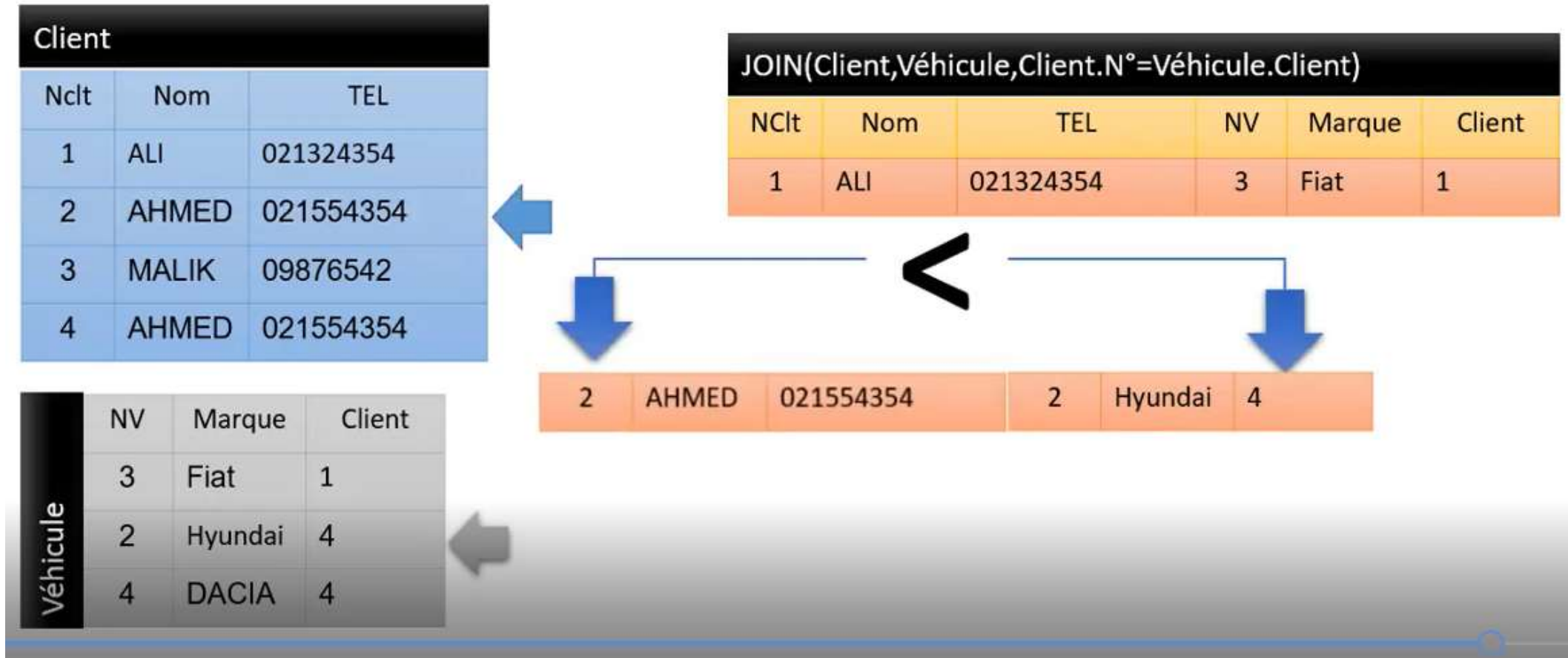
JOIN(Client,Véhicule,Client.N°=Véhicule.Client)					
NCl	Nom	TEL	NV	Marque	Client

1	ALI	021324354	3	Fiat	1
---	-----	-----------	---	------	---

Jointure par tri fusion



Jointure par tri fusion



Jointure par tri fusion

Client		
Nclt	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

Véhicule	NV	Marque	Client
	3	Fiat	1
	2	Hyundai	4
	4	DACIA	4

JOIN(Client,Véhicule,Client.N°=Véhicule.Client)					
NClt	Nom	TEL	NV	Marque	Client
1	ALI	021324354	3	Fiat	1



Jointure par tri fusion

Client		
Nclt	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

JOIN(Client,Véhicule,Client.N°=Véhicule.Client)					
NClIt	Nom	TEL	NV	Marque	Client
1	ALI	021324354	3	Fiat	1

Véhicule	NV	Marque	Client
	3	Fiat	1
	2	Hyundai	4
	4	DACIA	4

4	AHMED	021554354	2	Hyundai	4
---	-------	-----------	---	---------	---

Jointure par tri fusion

Client		
Nclt	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

JOIN(Client,Véhicule,Client.N°=Véhicule.Client)					
NClIt	Nom	TEL	NV	Marque	Client
1	ALI	021324354	3	Fiat	1
4	AHMED	021554354	2	Hyundai	4

Véhicule	NV	Marque	Client
	3	Fiat	1
	2	Hyundai	4
	4	DACIA	4

4	AHMED	021554354	4	DACIA	4
---	-------	-----------	---	-------	---



=



Jointure par tri fusion

Client		
Nclt	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

JOIN(Client,Véhicule,Client.N°=Véhicule.Client)					
NCl't	Nom	TEL	NV	Marque	Client
1	ALI	021324354	3	Fiat	1
4	AHMED	021554354	2	Hyundai	4
4	AHMED	021554354	4	DACIA	4

Véhicule	NV	Marque	Client
	3	Fiat	1
	2	Hyundai	4
	4	DACIA	4

Jointure par tri fusion

❑ Plus efficace que les boucles imbriquées pour les

grosses tables

❑ Principe

- Trier les deux tables sur les colonnes de jointure
- Effectuer la fusion
- Complexité: Le tri qui coûte cher

Coût =

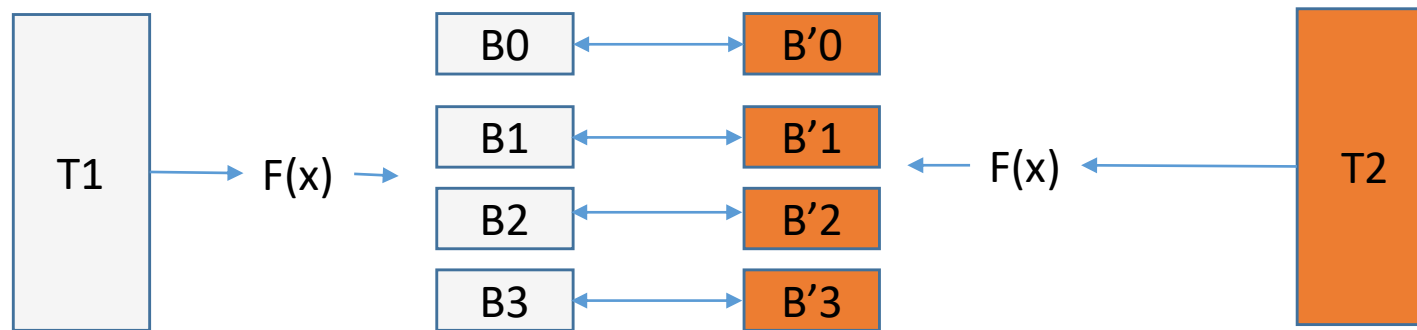
- $|T1| + |T2|$ si $T1, T2$ déjà triées
- $2 * |T1| * \log_b |T1| + |T2| + |T1|$ Si $|T2|$ triée
- $2 * |T2| * \log_b |T2| + |T2| + |T1|$ Si $|T1|$ triée
- $2 * |T1| * \log_b |T1| + 2 * |T2| * \log_b |T2| + |T2| + |T1|$ si aucune table n'est triée
- **b** : nombre de pages du tampon en mémoire centrale disponible pour le tri

Jointure par hachage

- 1.Hacher la plus petite des deux tables en n fragments
- 2.Hacher la seconde table, avec la même fonction, en n autres fragments
- 3.Réunir fragments par paire, et faire la jointure entre les fragments

Principe de la jointure par hachage

1. Hacher la plus petite des deux tables en n fragments
2. Hacher la seconde table, avec la même fonction, en n autres fragments
3. Réunir fragments par paire, et faire la jointure entre les fragments



Exemple : fonction de hachage Modulo 4

Coût jointure par hachage

1. Chargement de T1 : $|T1|$
 2. Hachage de T1, création des groupes en RAM
 3. Déchargement des groupes en MS : $|T1|$
 4. Chargement de T2 : $|T2|$
 5. Hachage de T2, création des groupes en RAM
 6. Déchargement des groupes en MS : $|T2|$
 7. Chargement du Block B_i de T1 avec B'_i de T2, faire la jointure entre les deux blocs : $|B_i| + |B'_i|$
 8. Au total : $\sum |B_i| + |B'_i| = |T1| + |T2|$
- Coût Total = $3(|T1| + |T2|)$

Jointure par hachage

- Le coût en nombre d'E/S: nombre de pages chargées pour pouvoir exécuter l'opération
- $3 * (|R| + |S|)$
- La plus petite des deux tables est divisée en N blocs, l'autre table sera divisée en N blocs
- Dans un premier temps on charge tous les blocs de R et on les remet
- $|R| + |R|$
- Ensuite on charge tous les blocs de S et on les remet
- $|S| + |S|$
- Pour finir on fait la jointure entre chaque blocs de R et de S
- $|R| + |S|$

- Les opérations dérivées
- Intersection
- Jointure externe
- Semi jointure
- Renommage

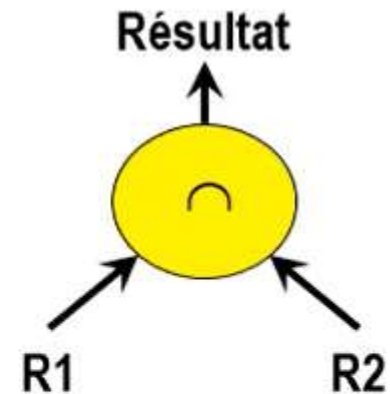
Intersection

Les deux relations doivent avoir la même structure, lorsque l'on souhaite trouver des éléments qui ont effectués deux actions en même temps, chaque élément de R1 est comparé avec R2, si c'est le même, on le prend

$$R_1 \cap R_2$$

$$INTERSECT (R_1, R_2)$$

$$AND (R_1, R_2)$$



$$R1 \cap R2 \equiv R1 - (R1 - R2)$$

Jointure externe (rappel thêta jointure)

Si on veut afficher tous les étudiants même ceux qui n'ont pas de notes et laisser la note à null, dans ce la thêta-jointure n'est pas suffisante, on utilise alors une jointure externe

Etudiants

student_id	name	age
1	Akon	17
2	Bkon	18
3	Ckon	17
4	Dkon	18

Notes

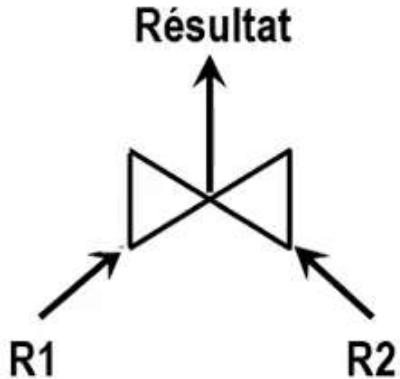


student_id	subject_id	marks
1	1	98
1	2	78
2	1	76
3	2	88

student_id	name	age	student_id	subject_id	marks
1	Akon	17	1	1	98
1	Akon	17	1	2	78
2	Bkon	18	2	1	76
3	Ckon	17	3	2	88

Jointure externe

3 types de jointure , gauche, toutes les ligne de R1 sont affich , droite toutes les ligne de R2 sont affich es, full, toutes les lignes sont affich e et celle qui satisfont la condition sont mise en correspondance

R1	R2	Pr�sentation	
XJoin(R1,R2)			
			
			
			

Jointure externe

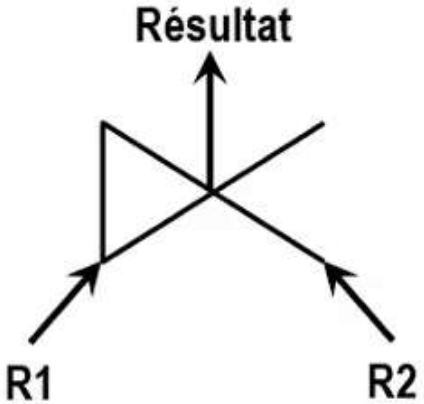
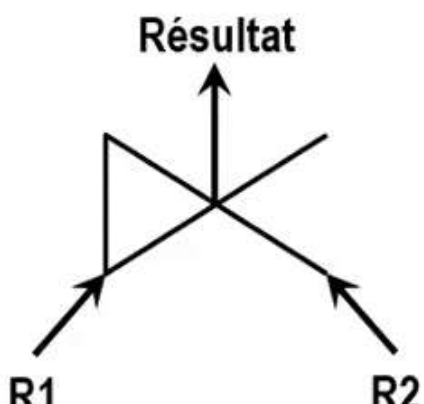
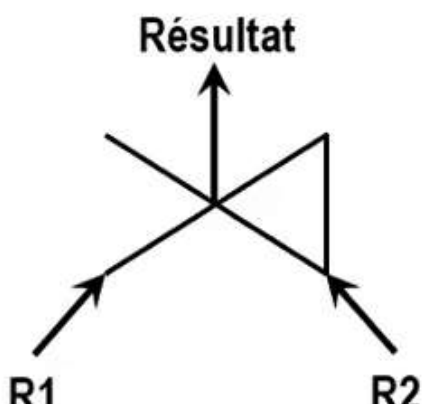
R1		
NCLT	Nom	TEL
1	ALI	021324354
2	AHMED	021554354
3	MALIK	09876542
4	AHMED	021554354

R2	NV	Marque	Client
	2	Hyundai	4
	3	Fiat	1
	4	DACIA	4

EXT-JOIN(R1,R2,NCl=Client)					
NCLT	Nom	TEL	NV	Marque	Client
1	ALI	021324354	3	Fiat	1
2	AHMED	021554354	-	-	-
3	MALIK	09876542	-	-	-
4	AHMED	021554354	2	Hyundai	4
4	AHMED	021554354	4	DACIA	4

Semi jointure

Jointure suivi d'une projection, afficher les éléments de jointure sans afficher les colonne de la deuxième relation

R1 R2	Présentation	
SJoin(R1,R2)		
		

Renommage

$\rho_{A \leftarrow B}(R)$	Présentation
Rename(R,Pos,Name)	Le résultat d'une expression algébrique ne possède pas de nom; On a besoin de renommer certains attributs d'une relation ou d'une expression algébrique. Le renommage permet de renommer une colonne d'une table dans la table résultat.

Les agrégats

- Count
- Sum
- Min
- Max
- Et on peut utiliser group by (pour chaque élément, on calcule ...)

Optimisation des requêtes

Une première s'appuyant sur les propriétés algébriques des opérateurs qui permet de limiter la taille des relations manipulées

Une seconde basée sur des fonctions de coût permet d'affiner l'optimisation en tenant compte des caractéristiques des relations et des attributs

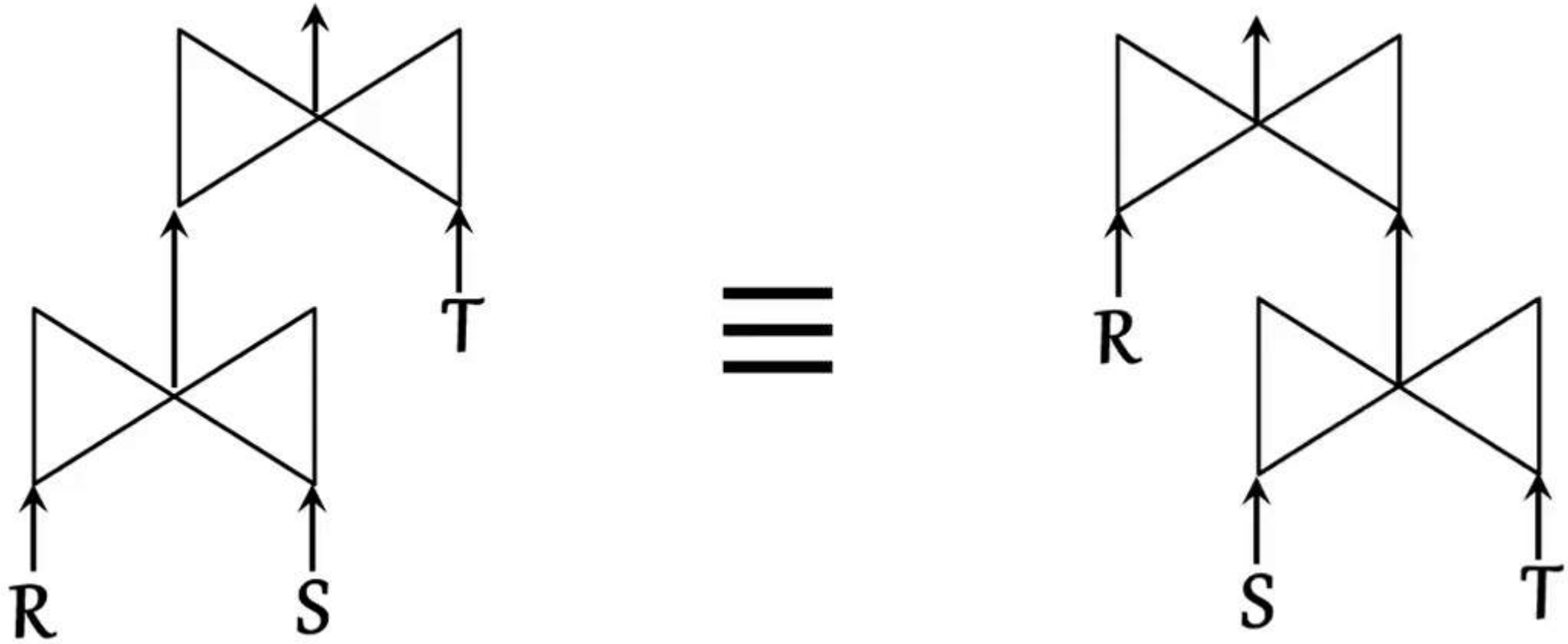
Optimisation en utilisant l'arbre algébrique

Il existe des propriétés permettant de modifier l'ordre des opérateurs algébriques, afin d'obtenir des gains de performance

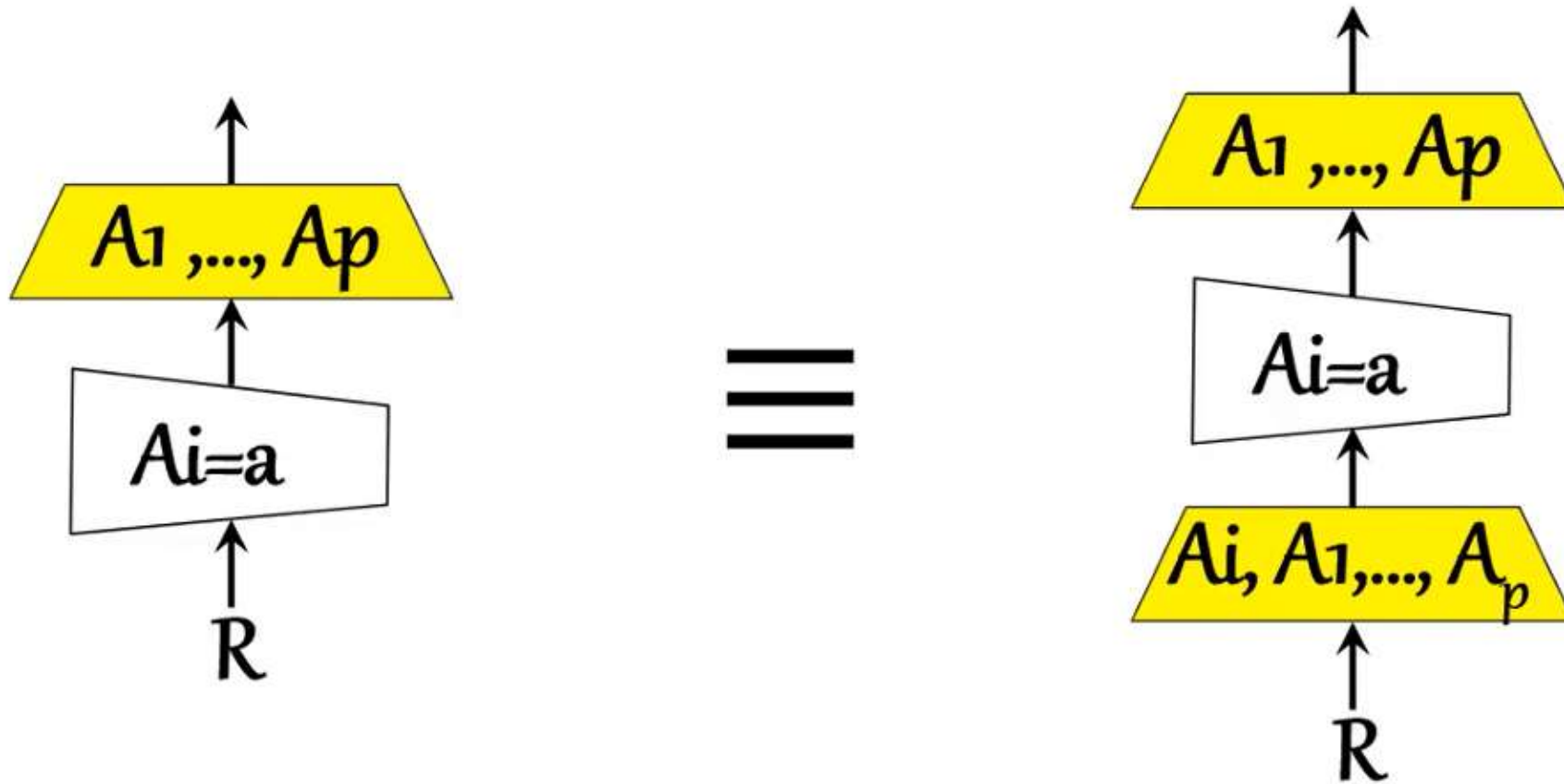
Règle 1: Commutativité de la jointure



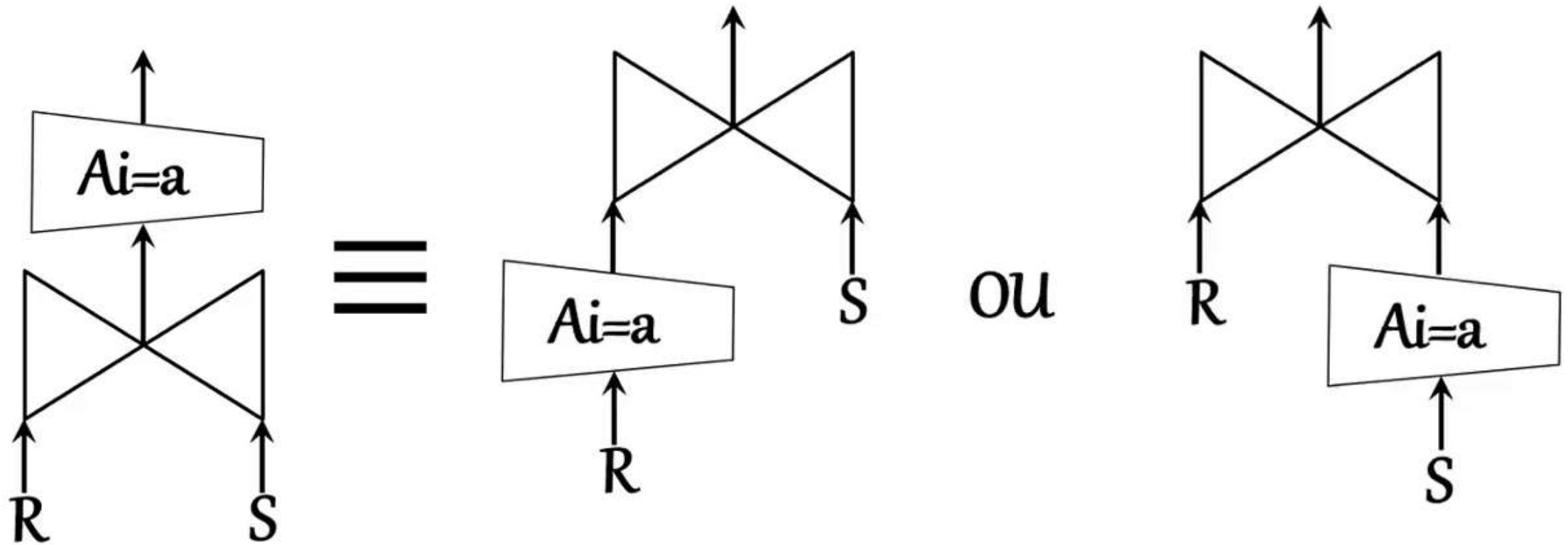
Règle 2: Associativité des jointures



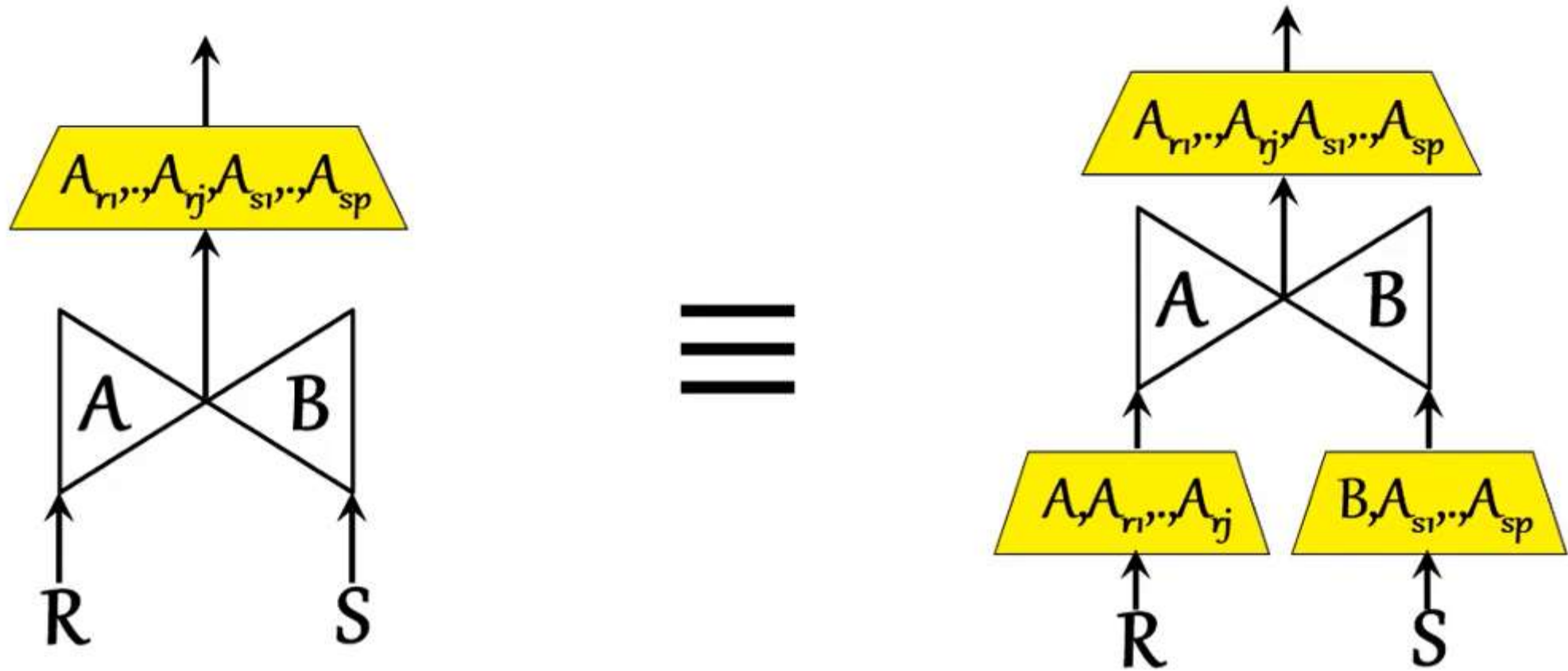
Règle 4: Semi commutativité des projections et restrictions



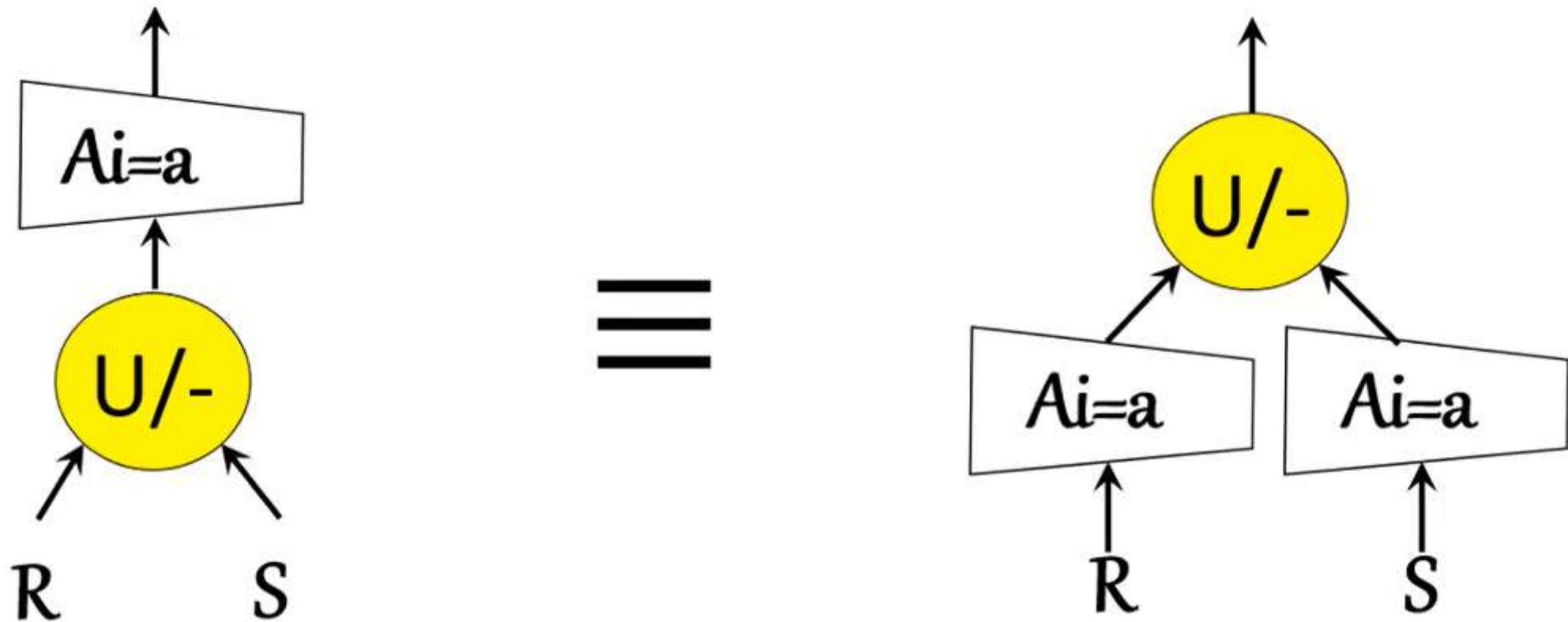
Règle 5: Distributivité des restrictions sur les jointures



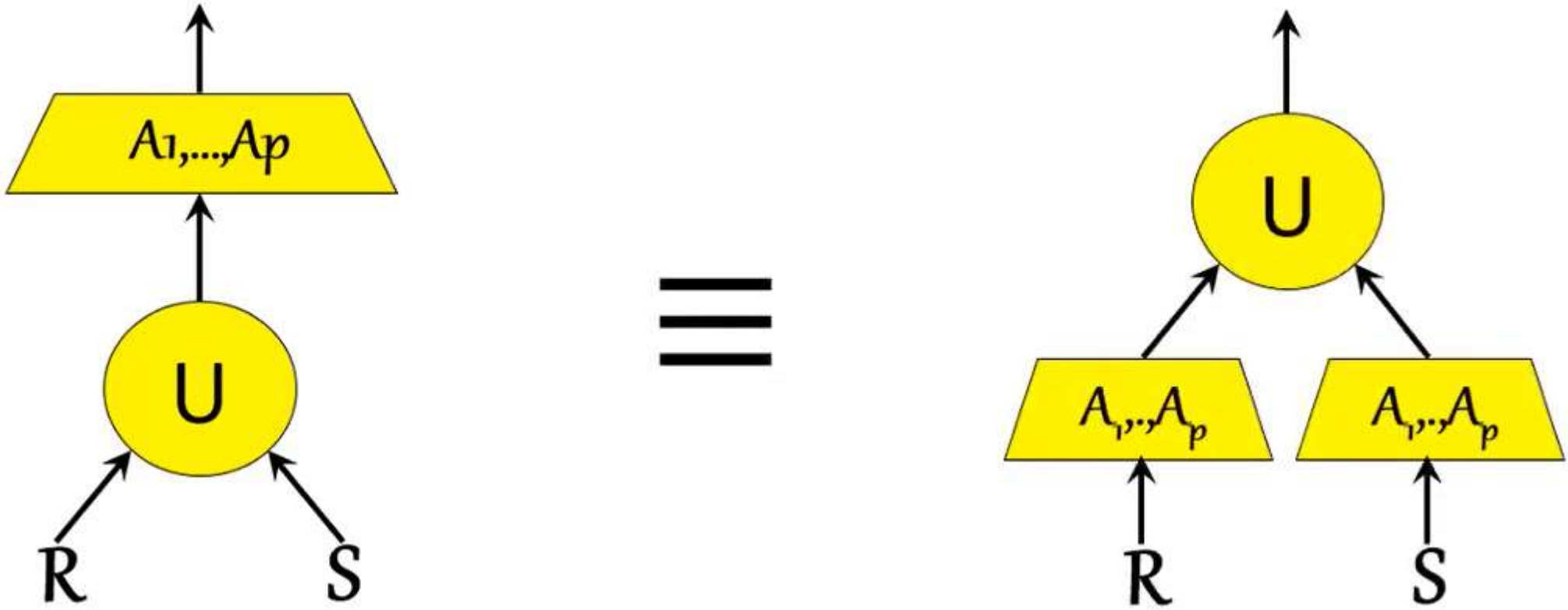
Règles 6: Semi distributivité des projections sur les jointures



Règle 7: Distributivité des restrictions sur l'union et la différence

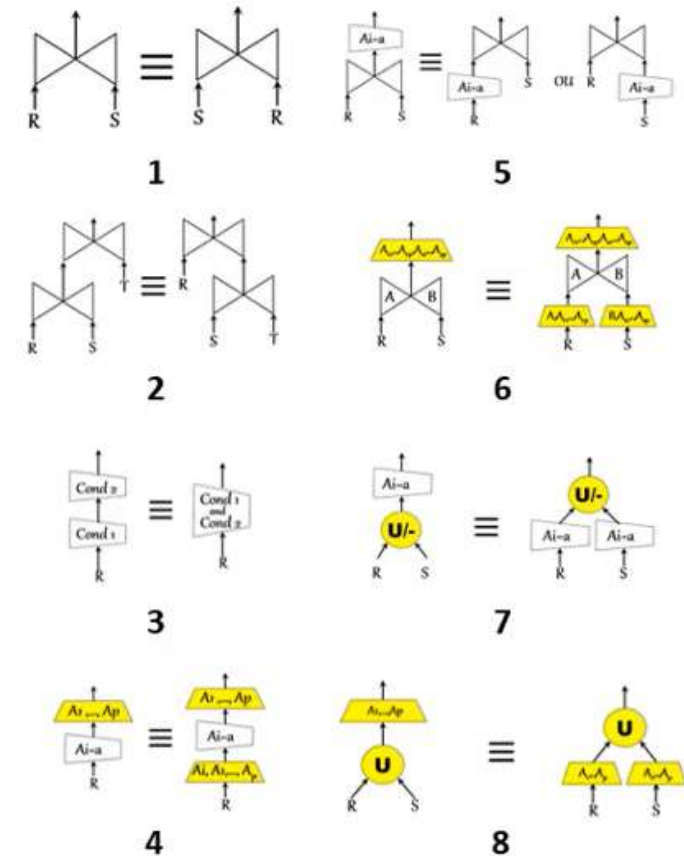


Règle 8: Distributivité des projections sur l'union

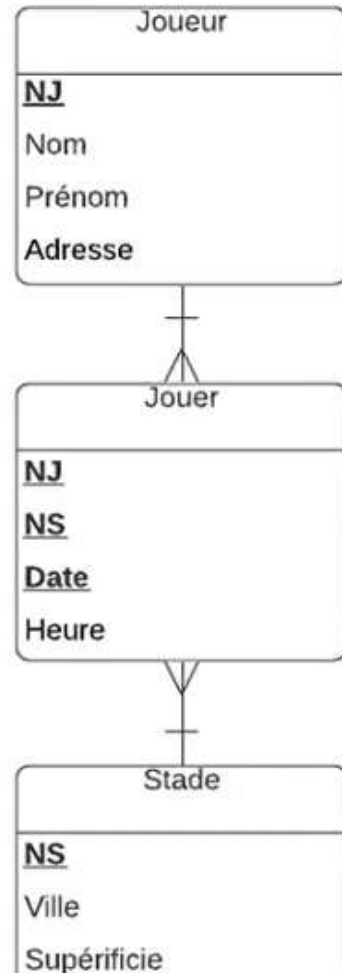


Application des règles pour l'optimisation

1	Décomposer les restrictions en restrictions Unaires	3
2	Rapprocher les restrictions des feuilles	4, 5 et 7
3	Grouper les restrictions aux feuilles	3
4	Rapprocher les projections des feuilles	4, 6 et 8
5	Ordonner les jointures afin de minimiser le temps	1 et 2



Exemple

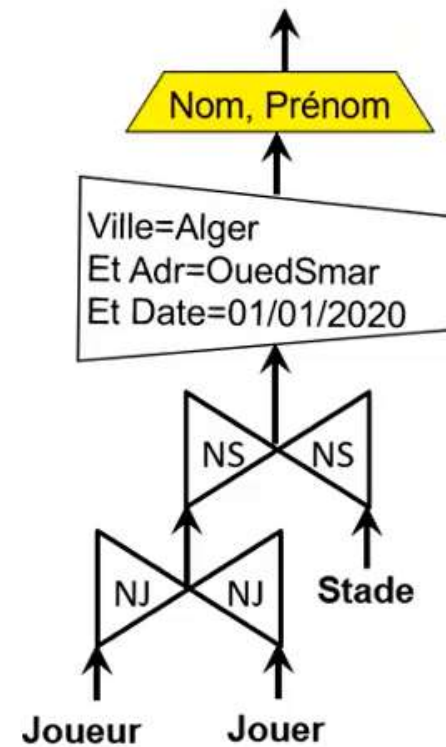
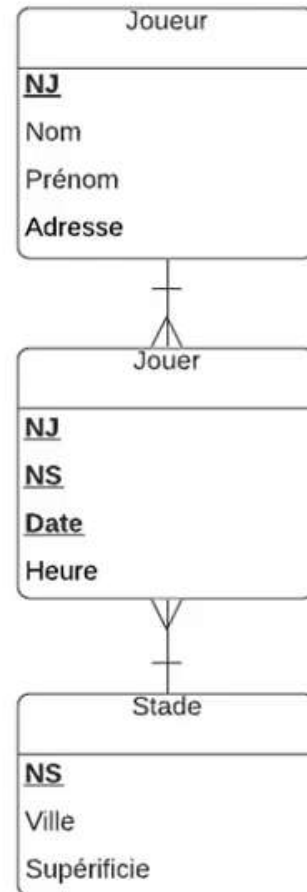


Liste des joueurs habitant à
OuedSmar, ayant joué un match dans
un stade d'Alger le 01/01/2020

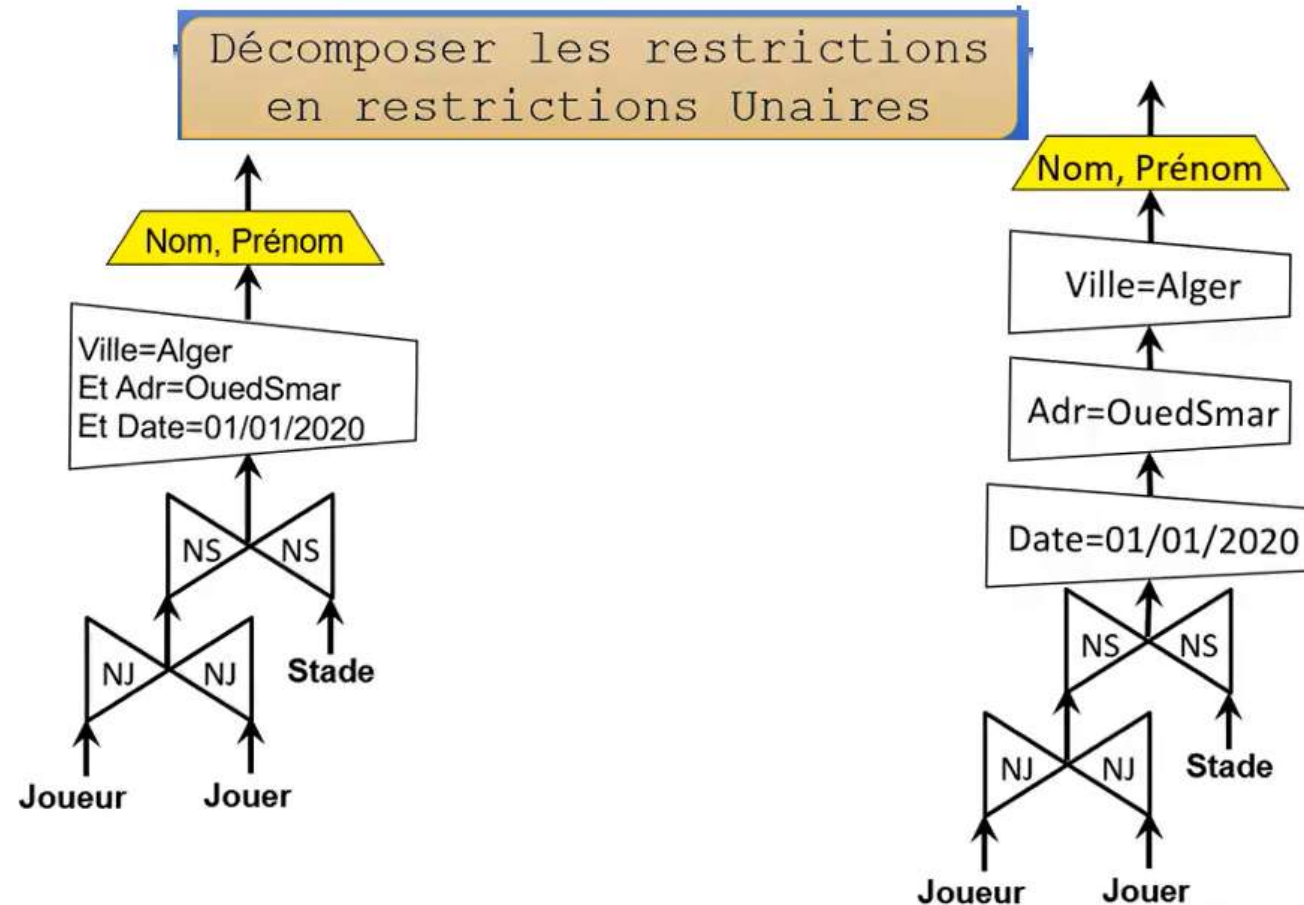
- Joueur (NJ, Nom, Prénom, Adresse)
- Stade(NS, Ville, Superficie)
- Jouer(NJ, NS, date, heure)
- Lister les joueur habitant Oued esemar, ayant joué un match dans un stade d'Alger le 01/01/2020

- Select nom, prénom (projection)
- From joueur j, jouer jo, stade s
- Where j.adresse= 'ouedessemar' (restriction)
- And s.ville = 'Alger' (restriction)
- And jo.date= 01/01/2020 (restriction)
- And J.nj= jo.nj (jointure)
- And jo.ns= s.ns; (jointure)

Exemple



1) Règle 3: décomposer les restrictions

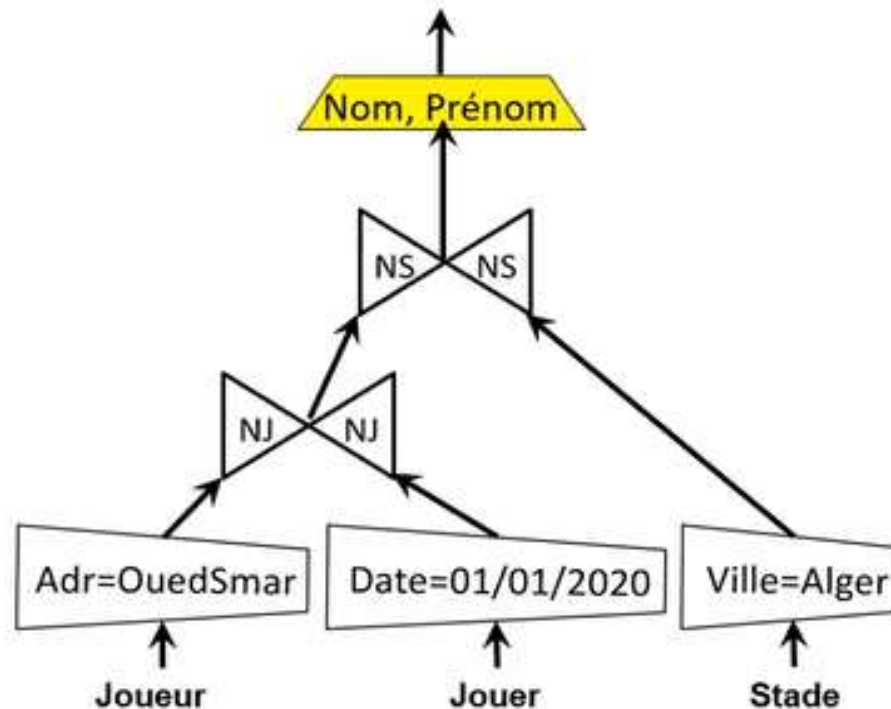
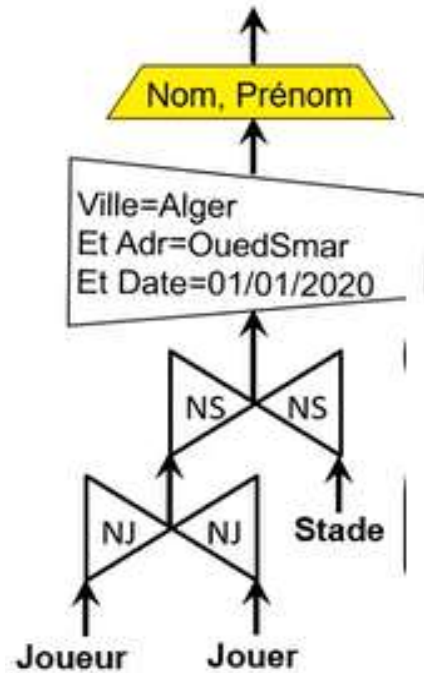


2) Règle 4, 5 et 7: faire descendre les restrictions sur leurs tables

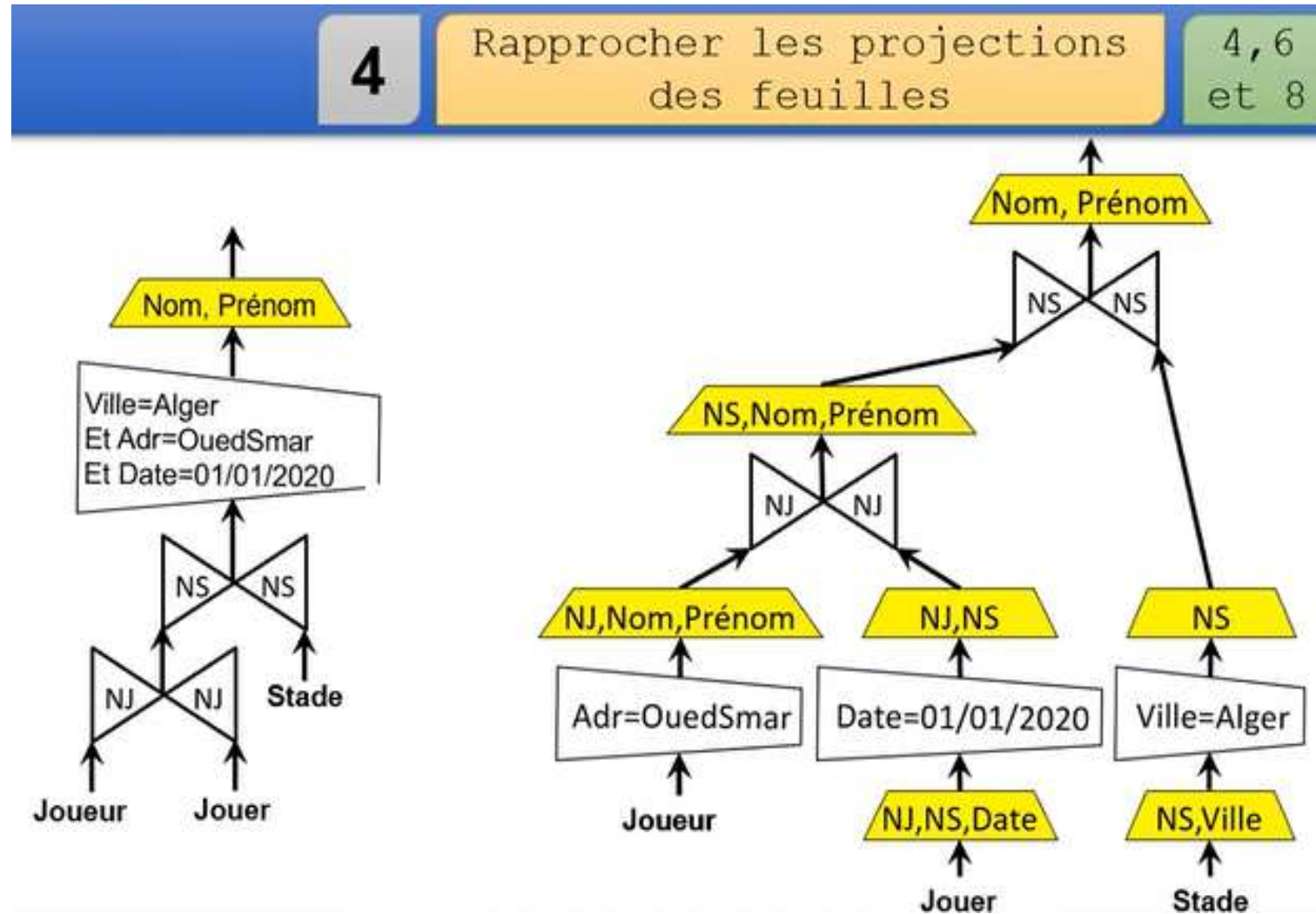
2

Rapprocher les restrictions
des feuilles

4, 5
et 7



4) Règles 4, 6 et 8: faire descendre les jointures le plus bas possible



Exercice 1

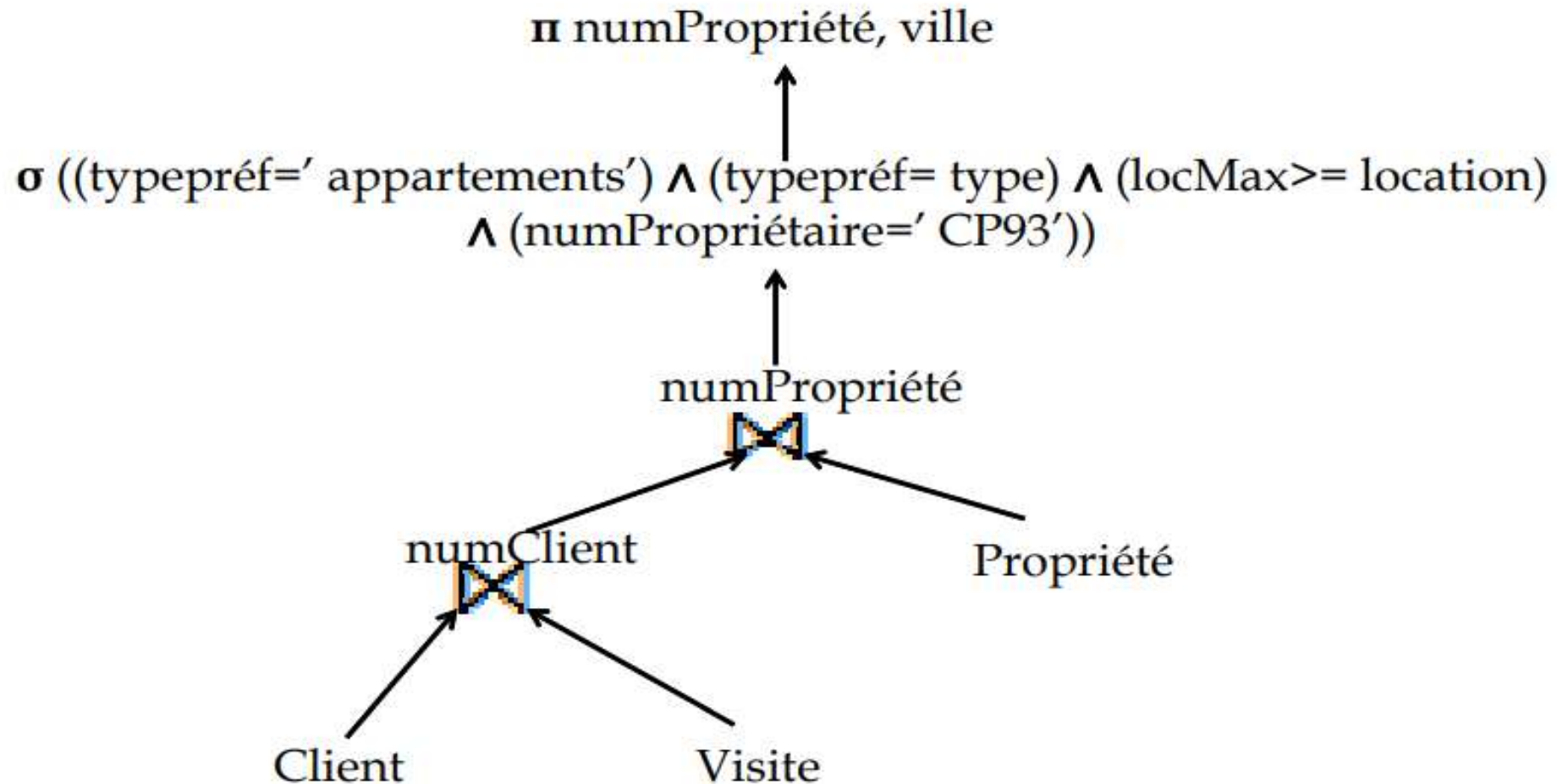
- Soit le schéma relationnel suivant :
- Propriétaire(numPropriétaire, prénom, nom, adresse, numtél).
- Propriété(numPropriété, ville, type, location, numPropriétaire*)
- Client (numClient, prénom, nom, adresse, typepréf, locMax)
- Visite (numPropriété*, numClient*, datevisite)
- a. Exprimer en SQL la requête suivante : pour le compte de locataires qui ont visité des appartements, trouver tous les logements qui correspondent à leurs exigences et qui appartiennent au propriétaire de numéro CP93.

Exercice 1

```
SELECT P.numPropriété, P.ville  
FROM Client C, Visite V, Propriété P  
WHERE typepréf=' appartements'  
AND typepréf= type  
AND locMax>= location  
AND numPropriétaire=' CP93'  
AND C. numClient=V. numClient  
AND V. numPropriété=P. numPropriété
```

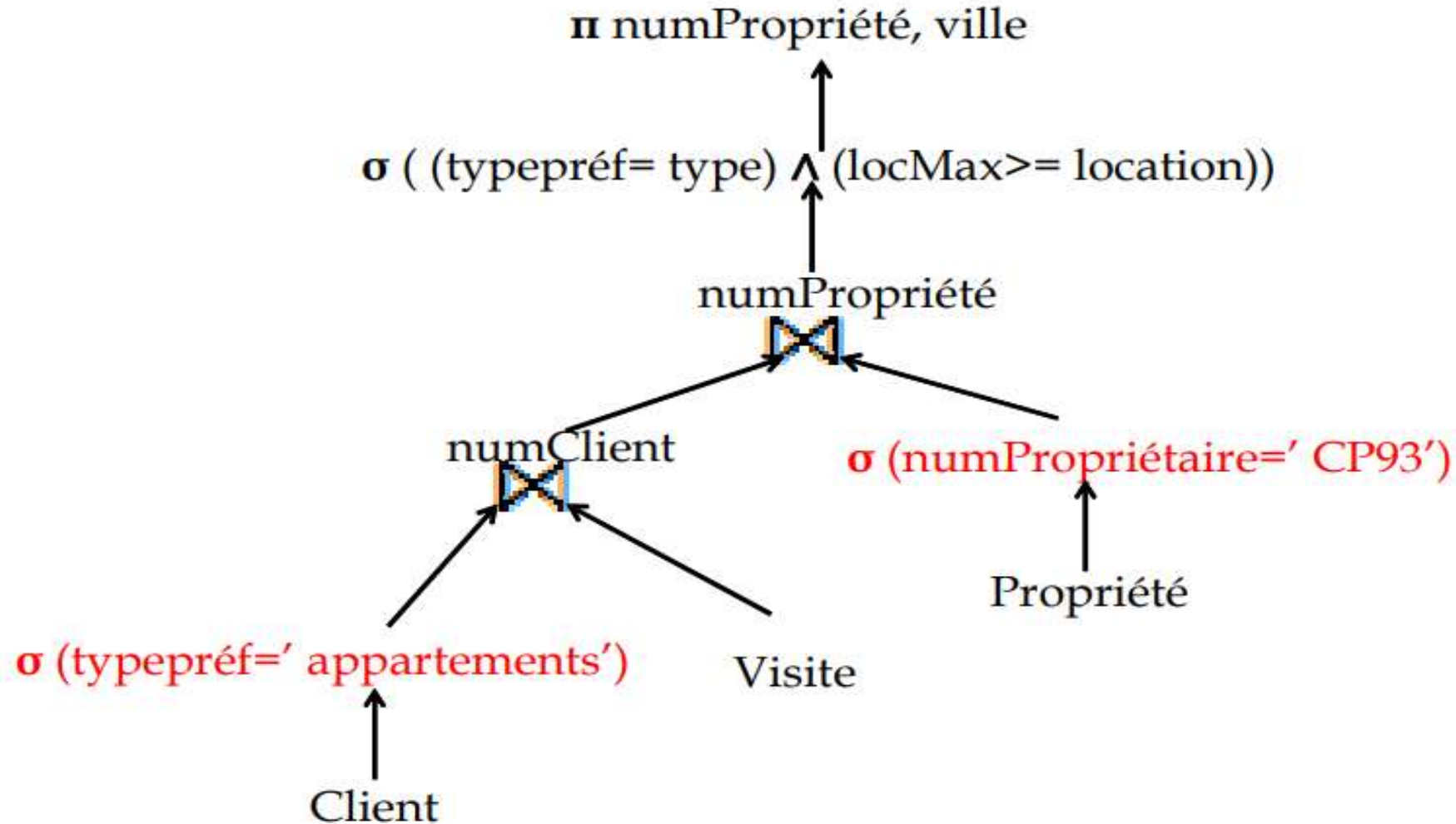
Exercice 1

- b. Donner l'arbre algébrique qui correspond à la requête SQL.



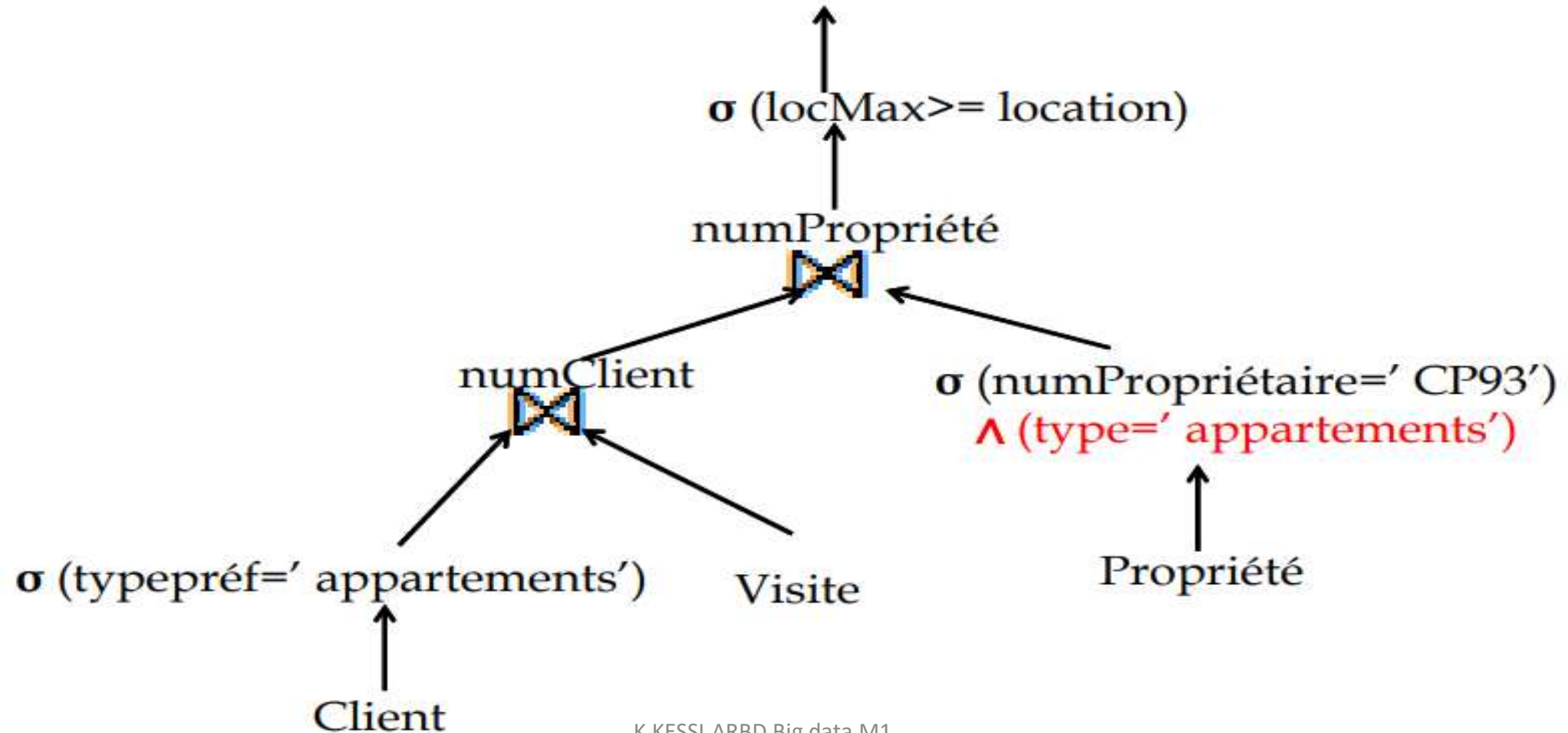
Exercice 1

- c. Optimiser la requête en utilisant les règles de transformation.
- Faire descendre les restrictions



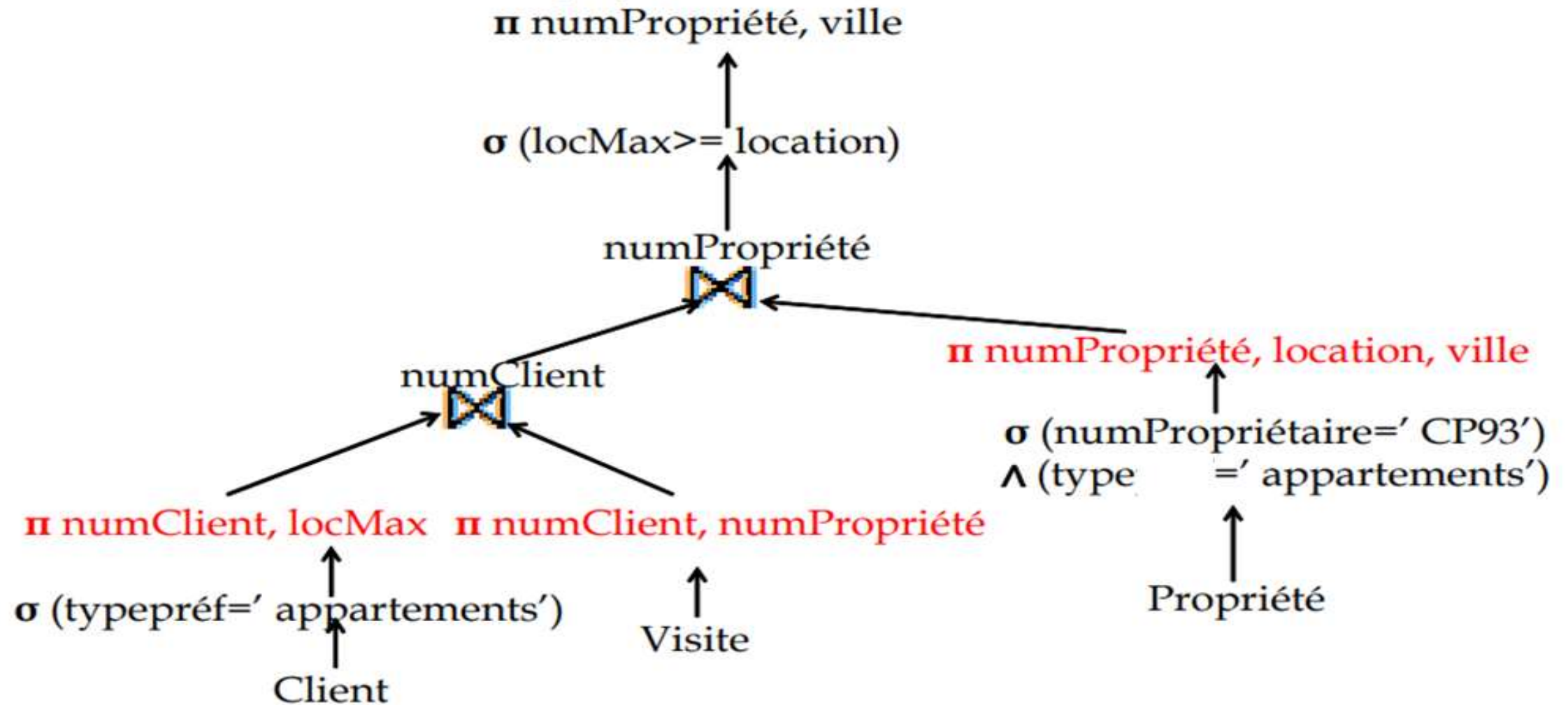
Exercice 1

- c. Optimiser la requête en utilisant les règles de transformation.
➤ Remplacer σ (typepré= type) par σ (type=' appartements')
- π numPropriété, ville



Exercice 1

- c. Optimiser la requête en utilisant les règles de transformation.
- Faire descendre les projections



- Méthode d'optimisation basée sur les coûts

Notions utiles

- ❑ Les tables relationnelles sont stockées physiquement dans des fichiers sur disque
- ❑ Lorsqu'on veut accéder aux données, on doit transférer le contenu pertinent des fichiers dans la mémoire
 - **Fichier** = séquence de tuples
 - **Bloc (page)** = unité de transfert de données entre disque et mémoire
 - **Facteur de blocage** = nombre de tuples d'une relation qui «tiennent» dans un bloc
 - **Coût de lecture (ou écriture) d'une relation** = nombre de blocs à lire du disque vers la mémoire (ou à écrire de la mémoire vers le disque)

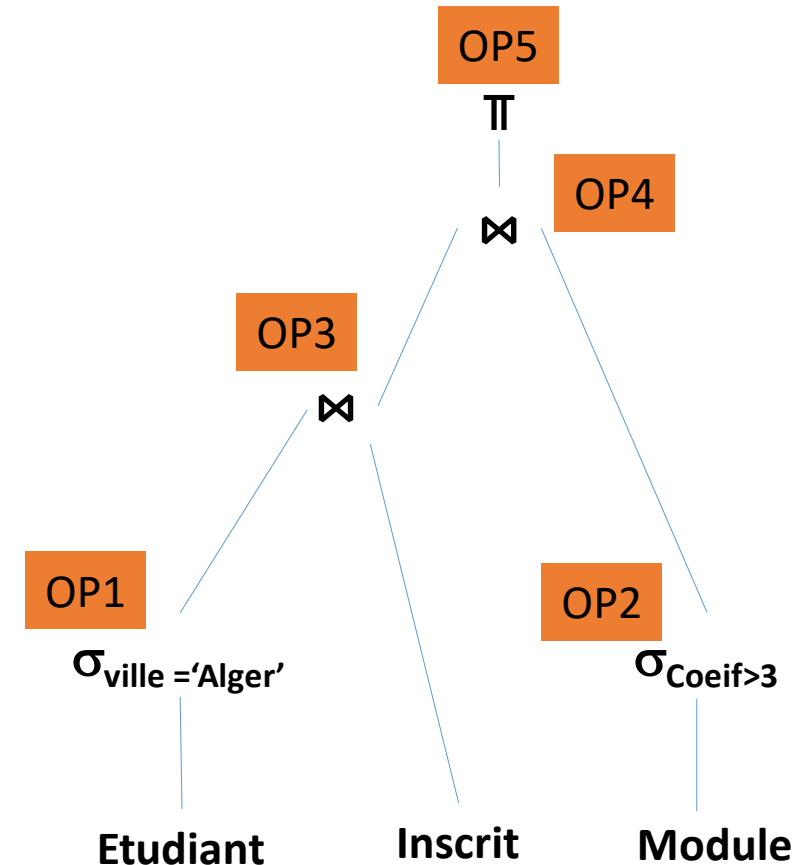
Statistiques au sujet des tables

Statistique	Signification
$ T $	Nombre de lignes de la table T
TailleLigne	La taille d'une ligne de la table T
FBT	Facteur de blocage moyen de T
NDIST (T,a)	Nombre de valeurs distinctes de la colonne a pour la table T
MinT(colonne)	Valeur minimum de la colonne de T
MaxT(colonne)	Valeur maximum de la colonne de T
$ T $	Nombre de pages de la table T

- Calculer le coût des opérations en 3 étapes:
- 1. Le coût en nombre d'E/S: nombre de pages chargées pour pouvoir exécuter l'opération
- 2. Le coût en nombre de tuples généré par l'opération
- 3. Le nombre de pages pour contenir les tuples générés par l'opération (le nombre de tuples de l'étape 2)
- (l'étape 1 est indépendante, l'étape 3 dépend de l'étape 2)

Coût d'une requête

- ❑ Soit R une requête composée de n opérations algébriques (O_{pi})
- ❑ $\text{Coût}(R) = \sum \text{Coût}(O_{pi})$
- ❑ $\text{Coût}(O_{pi}) = \text{Nombre de pages transférées de la mémoire secondaire vers la RAM ou de la RAM vers la mémoire SWAP}$
- ❑ Pour calculer le $\text{Coût}(O_{pi+1})$
 - Il faut calculer le nombre de tuples du résultat intermédiaire de O_{pi} ($||O_{pi}||$)
 - En utilisant le facteur de sélectivité de O_{pi} : $||O_{pi}|| = FS * ||O_{pi-1}||$
 - Le transformer en nombre de pages en utilisant les tailles du tuples (T_Tuples) et de la page système $||O_{pi}|| = T_tuples * ||O_{pi}|| / PS$
 - **Taille tuple** = $\sum$ taille attributs du tuple
 - **Une autre Formule** $||O_{pi}|| = ||O_{pi}|| / \#tuples \text{ par page}$
- ❑ Si RAM non suffisante : Ajouter les pages transférées en mémoire SWAP



- $T_Tuple(\sigma(T)) = T_Tuple(T)$
- $T_Tuple(T1 \bowtie T2) = T_Tuple(T1) + T_Tuple(T2)$
- $T_Tuple(\pi_{A1, A2, \dots}(T)) = \sum Taille(Ai)$

- $||(\sigma(R))|| = SF * ||R||$
- $SF(A = valeur) = 1 / NDIST(A)$
- $SF(P \text{ et } Q) = SF(P) * SF(Q)$
- $SF(A > valeur) = (\max(A) - valeur) / (\max(A) - \min(A))$